SIMATS ENGINEERING

ASSESSMENT TOOL 1

Constraint-Based Problem Solving – Level 1

Student Name	Paleru Dharani Govardhan
Register Number	192525280
Course Name	R Programming
Course Code	ITA04

Course Outcome Covered

CO: Apply R programming fundamentals including data structures, vectors, matrices, arrays, and class types to perform mathematical operations and manage data effectively.

Bloom Level: BL2–BL4

Assessment Tool Weightage: 20%

Total Marks: 5 × 20 = 100

QUESTION 1: Vectors: Weekly Sales Analysis

Aim: To create a named vector of daily sales and use vectorized operations and logical indexing to summarize weekly sales.

Algorithm:

1. 1. Create a named numeric vector using c() with the seven days as names.
2. 2. Calculate total sales using sum() and average sales using mean(), rounded to two decimal places.
3. 3. Use logical indexing to select sales greater than the weekly average.
4. 4. Use which.max() to find the day with the highest sales.

R Program:

```
sales <- c(Monday = 12, Tuesday = 18, Wednesday = 9,
           Thursday = 25, Friday = 30, Saturday = 14,
           Sunday = 22)

total_sales <- sum(sales)
average_sales <- round(mean(sales), 2)
above_average <- sales[sales > average_sales]
highest_sales_day <- names(sales)[which.max(sales)]

print(sales)
cat("Total sales:", total_sales, "\n")
cat("Average sales:", average_sales, "\n")
cat("Days above average:\n")
print(above_average)
cat("Day with highest sales:", highest_sales_day, "\n")
```

Output / Result:

```
Monday Wednesday  Thursday    Friday  Saturday    Sunday
     12         9       25        30        14        22
Total sales: 130
Average sales: 18.57
Days above average:
 Thursday    Friday    Sunday
       25        30        22
Day with highest sales: Friday
```

QUESTION 2: Lists: Student Record

Aim: To store a student's details in a list, calculate the average score, assign a grade, and inspect the list elements.

Algorithm:

5. 1. Create one list containing the name, marks vector, and passed status.
6. 2. Calculate the average using mean(student$marks).
7. 3. Assign the grade as a new list element using $; do not rebuild the list.
8. 4. Display the number of elements and their names.

R Program:

```
student <- list(
  name = "Asha",
  marks = c(78, 85, 92),
  passed = TRUE
)

average_marks <- mean(student$marks)
student$grade <- if (average_marks >= 85) "A" else "B"

cat("Average marks:", average_marks, "\n")
cat("Number of list elements:", length(student), "\n")
cat("Element names:", names(student), "\n")
print(student)
```

Output / Result:

```
Average marks: 85
Number of list elements: 4
Element names: name marks passed grade

$name
[1] "Asha"

$marks
[1] 78 85 92

$passed
[1] TRUE

$grade
[1] "A"
```

QUESTION 3: Matrices: Operations on Two Matrices

Aim: To create two 3 × 3 matrices and perform element-wise multiplication, matrix multiplication, transpose, and row-sum operations.

Algorithm:

9. 1. Create matrices A and B with matrix() and byrow = TRUE.
10. 2. Use * for element-wise multiplication.
11. 3. Use %*% for matrix multiplication.
12. 4. Use t() to transpose A and rowSums() to calculate its row totals.

R Program:

```
A <- matrix(1:9, nrow = 3, ncol = 3, byrow = TRUE)
B <- matrix(c(1, 0, 2, 0, 1, 0, 3, 0, 1),
            nrow = 3, ncol = 3, byrow = TRUE)

element_product <- A * B
matrix_product <- A %*% B
transpose_A <- t(A)
row_totals <- rowSums(A)

cat("Matrix A:\n"); print(A)
cat("Matrix B:\n"); print(B)
cat("Element-wise product:\n"); print(element_product)
cat("Matrix product A %*% B:\n"); print(matrix_product)
cat("Transpose of A:\n"); print(transpose_A)
cat("Row sums of A:\n"); print(row_totals)
```

Output / Result:

```
Matrix A:
     [,1] [,2] [,3]
[1,]    1    2    3
[2,]    4    5    6
[3,]    7    8    9

Matrix B:
     [,1] [,2] [,3]
[1,]    1    0    2
[2,]    0    1    0
[3,]    3    0    1

Element-wise product:
     [,1] [,2] [,3]
[1,]    1    0    6
[2,]    0    5    0
[3,]   21    0    9

Matrix product A %*% B:
     [,1] [,2] [,3]
[1,]   10    2    5
[2,]   22    5   14
[3,]   34    8   23

Transpose of A:
     [,1] [,2] [,3]
[1,]    1    4    7
```

```
[2,]    2    5    8
[3,]    3    6    9

Row sums of A:
[1] 6 15 24
```

QUESTION 4: Arrays: Three-Dimensional Data

Aim: To create a three-dimensional array and retrieve layer data and layer totals using indexing and apply().

Algorithm:

13. 1. Create an array containing values 1 through 12 with dimensions c(2, 3, 2).
14. 2. Extract the second layer using arr[, , 2].
15. 3. Extract row 2, column 3, layer 1 using arr[2, 3, 1].
16. 4. Use apply(arr, 3, sum) to calculate the sum of each layer.

R Program:

```
arr <- array(1:12, dim = c(2, 3, 2))

cat("Entire second layer:\n")
print(arr[, , 2])

cat("Row 2, column 3, layer 1:", arr[2, 3, 1], "\n")

layer_totals <- apply(arr, 3, sum)
cat("Layer totals:\n")
print(layer_totals)
```

Output / Result:

```
Entire second layer:
     [,1] [,2] [,3]
[1,]    7    9   11
[2,]    8   10   12

Row 2, column 3, layer 1: 6

Layer totals:
[1] 21 57
```

QUESTION 5: Class Types and Coercion: Price List

Aim: To inspect a character vector's class, convert its values to numeric, calculate the bill after a discount, and explain type coercion.

Algorithm:

17. 1. Create the provided prices as a character vector and display class().
18. 2. Convert the existing vector using as.numeric() and display its class.
19. 3. Calculate the total and apply a 10% discount.
20. 4. Create c(1, 'a', TRUE), inspect its class, and explain that R coerces the values to a common compatible type.

R Program:

```
prices_text <- c("120", "85.5", "200")
cat("Class before conversion:", class(prices_text), "\n")

prices <- as.numeric(prices_text)
cat("Class after conversion:", class(prices), "\n")

total_bill <- sum(prices)
discounted_total <- total_bill * 0.90
cat("Total bill:", total_bill, "\n")
cat("Bill after 10% discount:", discounted_total, "\n")

mixed_vector <- c(1, "a", TRUE)
print(mixed_vector)
cat("Class of mixed vector:", class(mixed_vector), "\n")
```

Output / Result:

```
Class before conversion: character
Class after conversion: numeric
Total bill: 405.5
Bill after 10% discount: 364.95
[1] "1"    "a"    "TRUE"
Class of mixed vector: character

Explanation:
R coerces all elements to a common type. Because the vector contains
a character value ("a"), the numeric value 1 and logical value TRUE
are converted to character strings. Therefore, the resulting vector
has class "character".
```

Code of Conduct

I, Paleru Dharani Govardhan (Register No. 192525280), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

Date: _____

Assessment Rubric (Provided Criteria)

Criteria	Weightage	Level 4 – Exemplary	Levels 3 to 1
Problem Understanding	20%	Identifies all inputs, outputs, and constraints accurately.	L3: Minor missing elements; L2: Partial understanding; L1: Incorrect interpretation.
Algorithm Design	30%	Complete, correct, logically sound solution.	L3: Minor logical gaps; L2: Partially correct logic; L1: Incorrect approach.
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules.	L3: Minor inefficiencies; L2: Limited/basic usage; L1: Incorrect or missing

			constructs.
Pseudocode Structure	10%	Well-structured, clear, consistent, easy to follow.	L3: Mostly clear with minor issues; L2: Difficult to follow in parts; L1: Poorly structured/unclear.
Completeness	20%	Handles all cases with an optimized approach.	L3: Handles most cases; L2: Basic cases only; L1: Incomplete or inefficient.